

本模块实现了计算由漫反射材质和自发光材质得到光线颜色；实现球体、四边形和均匀介质等三维物体的求交计算；通过包围盒和BVH 树加速求交的计算。

Material 类均用 emitted 函数返回自发光材质的颜色，scatter 函数返回漫反射材质的散射情况。多种类型的材质虽然计算方法各不相同，但都会返回这两个参数，因此我们就可以很方便的在 camera.cpp 中直接用这两个参数参与运算，而不用分类讨论材质的情况。ray_color 函数是一个递归函数，如果发生散射，散射颜色 = 衰减颜色 * 递归计算下一条光线的颜色。达到递归上限后，光线颜色为黑色（衰减到没有了）。

```
vecf3 Camera::ray_color(const Ray& r, int depth, const Hittable& world) const {
    // 达到递归深度上限，返回黑色，表示光线能量耗尽
    if (depth <= 0)
        return vecf3(0, 0, 0);

    Record rec; // 创建记录相交信息的对象

    // 1) 判断 world 是否与 r 相交，如果不相交则返回背景颜色
    if (!world.hit(r, Interval(0.001, infinity), rec)) {
        return background;
    }

    // 2) 直接计算自发光材质的颜色
    vecf3 emitted_color = rec.mat->emitted(rec.u, rec.v, rec.p); // 获取材质在相交点发出的颜色

    // 3) 判断是否散射，并得到散射光线和衰减颜色
    vecf3 attenuation; // 衰减颜色
    Ray scattered; // 散射光线
    if (!rec.mat->scatter(r, rec, attenuation, scattered)) { // 如果不发生散射
        return emitted_color; // 返回自发光颜色
    }

    // 4) 如果发生散射
    // 递归计算下一条光线的颜色，并乘以衰减颜色
    vecf3 scattered_color = ray_color(scattered, depth - 1, world).cwiseProduct(attenuation);
    //vecf3 scattered_color = ray_color(scattered, depth - 1, world).cwiseQuotient(attenuation);
    // 返回散射颜色与自发光颜色的和
    return scattered_color + emitted_color;
}
```

求交计算中, 比较重要的是光线的表示: $r(t)=O+td$, 通过联立可以求得 t , $r.at(t)$ 表示交点的位置, 再进一步计算相交情况。注意通过 6 个正方形构成一个立方体时, `make_shared<Quad>` 的参数传入 q, u, v, mat , u, v 叉乘得到的法向量方向指向立方体的外侧。实现的过程采用了先表示立方体的 8 个顶点的方法, 再用顶点相减得向量。

```
// Returns the 3D box (six sides) that contains the two opposite vertices a & b.
std::shared_ptr<HittableList> box(const vecf3& a, const vecf3& b, std::shared_ptr<Material> mat) {
    auto sides = std::make_shared<HittableList>();

    // TODO: Use the Quad class to create the six sides of the box.
    // `a` and `b` are the two opposite vertices of the box.
    // 计算立方体的六个面的顶点
    vecf3 p0 = a;
    vecf3 p1 = vecf3(b.x(), a.y(), a.z());
    vecf3 p2 = vecf3(b.x(), b.y(), a.z());
    vecf3 p3 = vecf3(a.x(), b.y(), a.z());
    vecf3 p4 = vecf3(a.x(), a.y(), b.z());
    vecf3 p5 = vecf3(b.x(), a.y(), b.z());
    vecf3 p6 = b;
    vecf3 p7 = vecf3(a.x(), b.y(), b.z());

    // 底面
    sides->add(std::make_shared<Quad>(p0, p1 - p0, p3 - p0, mat));
    // 上面
    sides->add(std::make_shared<Quad>(p5, p4 - p5, p6 - p5, mat));
    // 右面
    sides->add(std::make_shared<Quad>(p0, p3 - p0, p4 - p0, mat));
    // 左面
    sides->add(std::make_shared<Quad>(p1, p5 - p1, p2 - p1, mat));
    // 后面
    sides->add(std::make_shared<Quad>(p3, p2 - p3, p7 - p3, mat));
    // 前面
    sides->add(std::make_shared<Quad>(p0, p4 - p0, p1 - p0, mat));

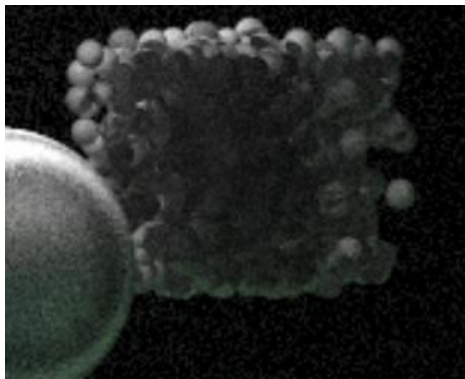
    return sides;
}
```

在 `ConstantMedium` 中, 注意 `boundary->hit`, 需要检查光线与边界在整个路径上的交点, 即从负无穷到正无穷的范围, 否则可能会出现函数始终 `return false`, 蓝球变透明的情况。

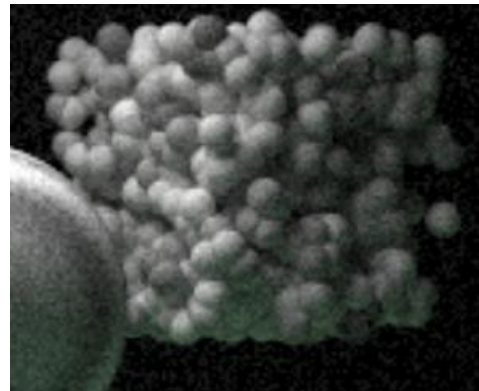
BVH 算法模块还有一个细节，如图，在递归地判断左子树和右子树是否有交点的中间，插入一个更新 `ray_t.max` 的语句，确保后续的判断是准确的。

```
bool BVHNode::hit(const Ray& r, Interval ray_t, Record& rec) const {  
    // 判断光线与当前节点的AABB是否相交  
    if (!bbox.hit(r, ray_t)) {  
        return false;  
    }  
  
    // 递归地判断与左BVH结点相交  
    bool hit_left = left->hit(r, ray_t, rec);  
  
    // 如果左子树有交点，则更新射线参数区间，再递归地判断与右BVH结点相交  
    if (hit_left) {  
        ray_t.max = rec.t;  
    }  
    bool hit_right = right->hit(r, ray_t, rec);  
  
    return hit_left || hit_right;  
}
```

事实上，如果没有这个语句，立方体的光影效果会出现一点问题。

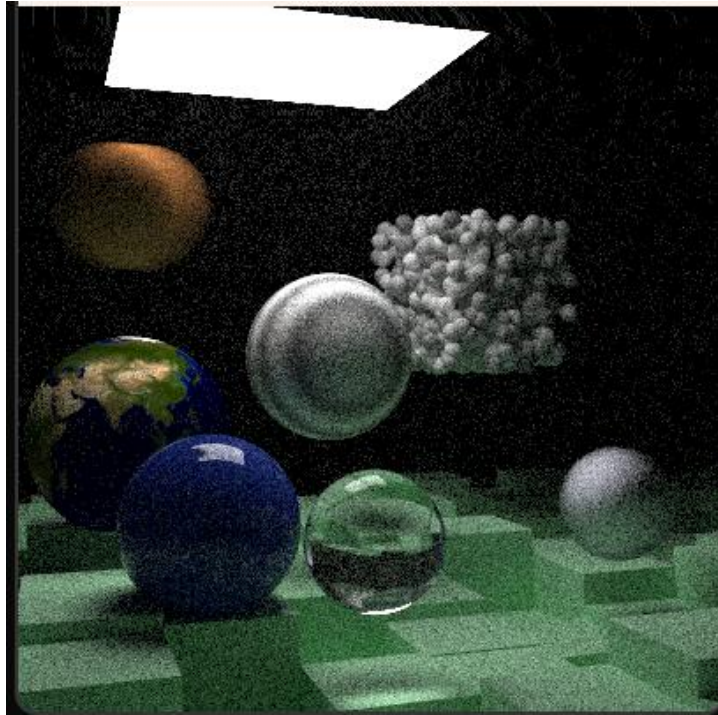


无更新

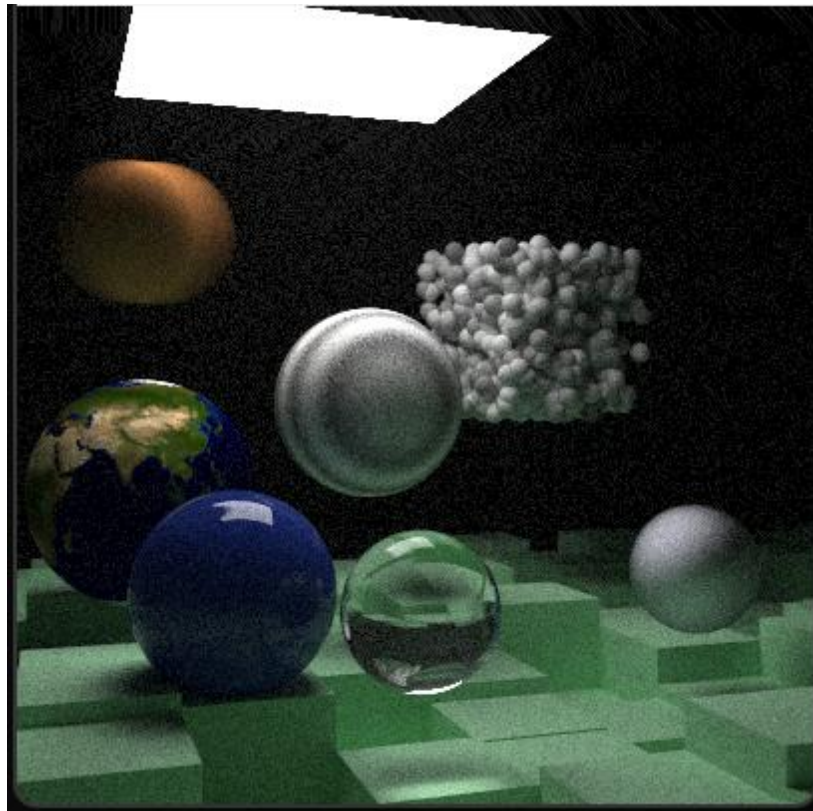


有更新

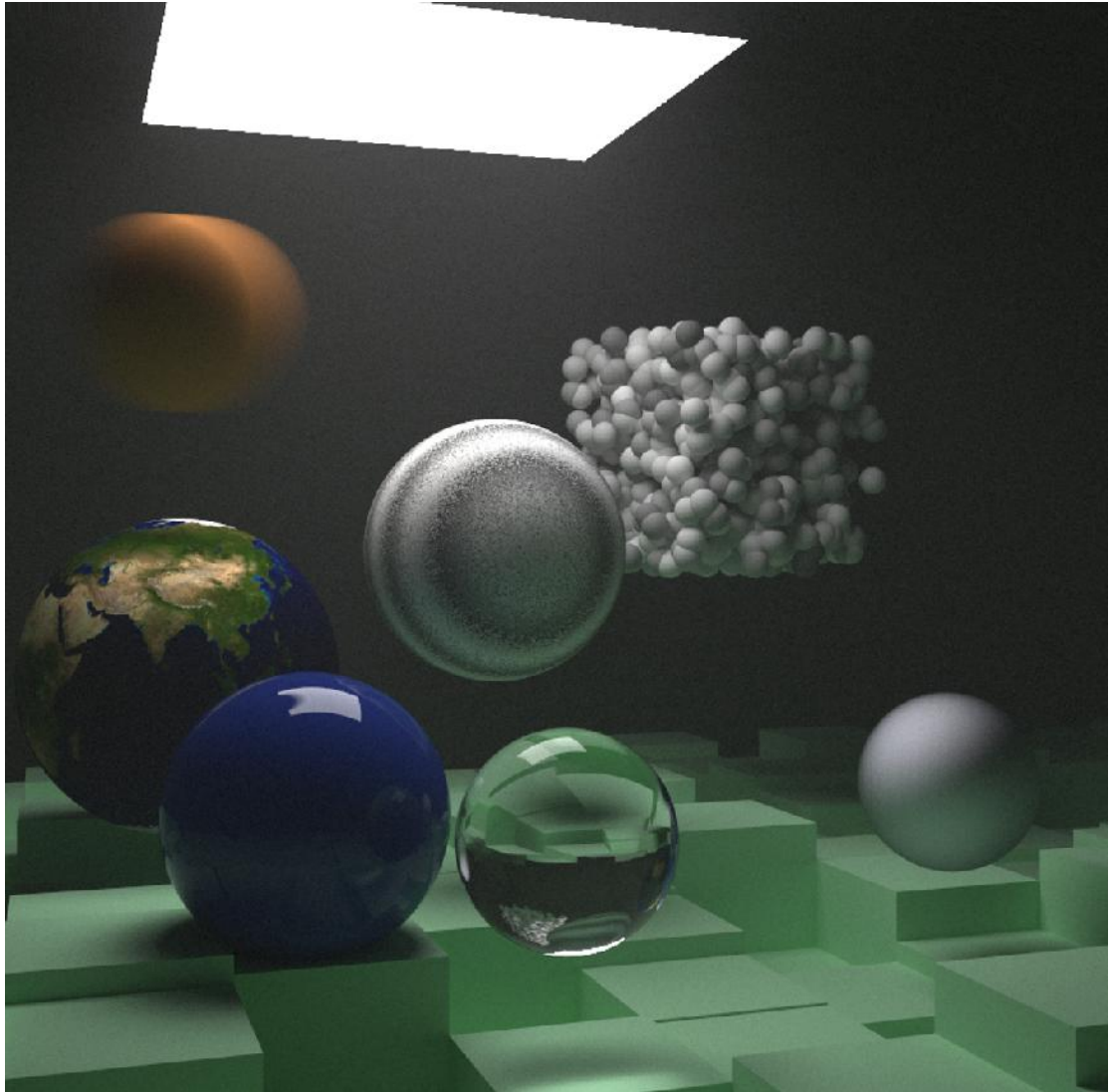
效果展示



400*400, 采样次数 250



400*400, 采样次数 1000



800*800, 采样次数 10000

